SyncQuest

Technical Deep Dive

Every Part 2 mechanism, file and message explained -- Distributed Systems Project, Part 2

This is the Part 2 companion to the Technical Deep Dive for Part 1 and to the Part 2 project report. Where the report explains what vector clocks, Bully election and the hash-chain audit log do, this document explains how they are actually built: file by file, hook by hook, message by message. It also separates what was **tested** from what was found only by **reading the code**, so edge cases are not presented as verified facts.

1. What Part 2 Added to the Layout

Part 2 added new files and changed only App.jsx; Part 1's files are unchanged in role. The new code is split into *pure logic* (no React, no network) under `src/sync/`, and *hooks* that connect that logic to Yjs awareness and React state.

```
frontend/src/
  sync/                        # NEW - pure, framework-free logic
    vectorClock.js             # create/clone/increment/merge/compare
    hashChain.js               # genesis, SHA-256 entry hashing, verifyChain
    originUtils.js             # isRemoteUpdateOrigin(origin, provider)
  hooks/
    useVectorClockLog.js       # NEW - Feature 1
    useCoordinatorElection.js # NEW - Feature 2 (Bully)
    useAuditLog.js             # NEW - Feature 3
  components/
    DebugPanel.jsx             # NEW - Vector Clocks + Election Log tabs
    CoordinatorBadge.jsx       # NEW - crown badge + force re-election
    AuditLogPanel.jsx          # NEW - chain viewer + tamper button
  App.jsx                      # MODIFIED - calls the three new hooks
```

The dependency direction is one-way: `sync/*` imports nothing; hooks import `sync/*`; `App.jsx` imports hooks; components only receive props. The three Part 2 hooks do not import each other. They are coupled only through values `App.jsx` passes in: the election hook's `coordinatorId` and `isCoordinator` are inputs to the audit hook.

1.1 Order of hooks in RoomApp

```
const { doc, provider, status, peers, ready } = useYjsRoom(roomId, identity)   // Part 1
const toasts = useToasts(peers, status)                                        // Part 1 cosmetic
useVectorClockLog(doc, provider, ready, identity)                              // F1
useCoordinatorElection(doc, provider, ready, identity, peers)                  // F2
useAuditLog(doc, provider, ready, identity, peers, coordinatorId, isCoordinator) // F3
```

Every Part 2 hook is gated on `ready` (the y-indexeddb 'synced' event from Part 1). This matters for vector clocks: y-indexeddb applies stored updates with its own object as origin, which `isRemoteUpdateOrigin` would classify as local. Since the Part 2 listeners attach only after the stored state is loaded, restoring a saved document on refresh is never counted as a user edit.

2. The Awareness Side-Channel, Precisely

Part 1 already used Yjs awareness for presence. Part 2 reuses the same object, `provider.awareness`, for all of its messages. Understanding its exact semantics explains most design decisions in the three hooks.

2.1 The wire path

- A hook calls `awareness.setLocalStateField(field, value)`. Awareness merges the field into this client's single JSON state object.
- Awareness emits `'update'` synchronously. y-webrtc's handler (`_awarenessUpdateHandler`) encodes that client's state and broadcasts it immediately to every connected peer as a type-1 message, so each call is its own message, not batched.
- On the receiving side `applyAwarenessUpdate` replaces the sender's whole state, and emits `'change'` with `{added, updated, removed}` only when the state is deep-different from what was stored.
- Awareness is not persisted and not CRDT-merged: last value wins per client. A client's entry disappears on clean exit (y-webrtc removes it on window unload) or, after a crash, once

`outdatedTimeout` (30 s) passes without an update.

2.2 The consequence every hook has to handle

A peer's awareness state is one object holding *all* of its fields at once. Remote state looks like this:

```
states.get(clientId) = {
  user:             { name, color, avatar, className, cursor... },        // Part 1
  vc:               { clock: { '123': 4, '456': 2 }, ts: 1728500000 },    // F1
  priority:         123,                                                  // F2
  coordMsg:         { type:'ALIVE', from:'123', term:2, seq:7, to:'456' },// F2
  knownCoordinator: { coordinatorId:'123', term:2 },                      // F2
  auditEntry:       { entry:{...}, seq:3 }                                // F3
}
```

Whenever *any* field changes (for example the peer simply moves its text cursor, a field Part 1 owns), a `'change'` event fires for that peer, and its old `vc`, `coordMsg` and `auditEntry` are still sitting in the object. A naive listener would replay a one-shot message every time a cursor moved. This is why each hook de-duplicates by something monotonic that the sender controls:

Hook	De-dup key	Where stored
useVectorClockLog	`vc.ts` (timestamp of the edit)	`seenRemoteTsRef: Map<peerId, ts>`
useCoordinatorElection	`coordMsg.seq` per sender	`processedRef: Map<peerId, lastSeq>`
useAuditLog	`auditEntry.seq` per sender	`processedRef: Map<peerId, lastSeq>`

2.3 Persistent state vs. one-shot messages

Awareness fits two different usages, and the election hook uses both. `priority` and `knownCoordinator` are **state**: any peer, including one that joins later, can read them at any time. `coordMsg` is a **message**: it is only meaningful the first time it is seen, which the `seq` check enforces.

3. Vector Clocks: sync/vectorClock.js

The module is a handful of pure functions over plain objects of the form `{ [peerId]: counter }`, where `peerId = String(doc.clientID)`. Missing keys count as 0.

incrementClock(c, id)	Returns a new object with `c[id] + 1`; never mutates the input.
mergeClocks(a, b)	Component-wise max over the union of keys.
compareVectorClocks (a, b)	Walks the union of keys, tracking whether a is below b somewhere and b below a somewhere. Neither: `'equal'`. Only a below: `'before'`. Only b below: `'after'`. Both: `'concurrent'`.
cloneClock / createEmptyClock	Immutability helpers so React state and refs never alias.

3.1 Worked comparisons

Clock a	Clock b	a below b somewhere?	b below a somewhere?	Result
{A:1}	{B:1}	yes (B: 0 < 1)	yes (A: 0 < 1)	concurrent
{A:2}	{A:2, B:2}	yes (B)	no	before
{A:3, B:2}	{A:2, B:2}	no	yes (A)	after
{A:2, B:2}	{A:2, B:2}	no	no	equal

3.2 The hook: useVectorClockLog.js

Two listeners are attached once `ready` is true.

```
// local edit
doc.on('update', (_u, origin) => {
  if (isRemoteUpdateOrigin(origin, provider)) return      // ignore remote edits
  const next = incrementClock(clockRef.current, localPeerId)
  clockRef.current = next
  awareness.setLocalStateField('vc', { clock: next, ts: Date.now() })
  appendEntry(localPeerId, identity.name, next, ts)
})

// remote edit, learned through awareness
awareness.on('change', ({added, updated}) => { ...
  if (seenRemoteTsRef.get(peerId) === vc.ts) return        // de-dup
  clockRef.current = mergeClocks(clockRef.current, vc.clock)
  appendEntry(peerId, name, vc.clock, vc.ts)
})
```

Details that matter:

- **Remote detection by origin.** y-webrtc applies remote updates with `Y.applyUpdate(doc, update, room)`, so `origin === provider.room` identifies a remote edit with no change to the Yjs wire format.
- **The clock travels on awareness, not with the document update.** A receiving peer learns about a remote edit's clock from the `vc` field, a separate message from the Yjs update. The clocks are a record of what was *broadcast*, not a re-derivation from Yjs's internal state.
- **Relation is to the previous log row.** `appendEntry` compares each new clock with `lastShownClockRef` (the clock of the entry most recently appended to the panel), not with every earlier edit. The panel answers "how does this event relate to the one just above it?", which is what a

live demo needs.

- **Bounded memory.** The log is capped at 60 entries (`MAX_LOG_ENTRIES`), newest first.
- **Why concurrency shows up at all.** When two peers type within about a second of each other, each increments its own counter before the other's broadcast has been merged, so neither clock dominates the other and the comparison returns `'concurrent'`.

4. Bully Election: useCoordinatorElection.js

This is the largest hook (233 lines). It is a small state machine driven by four inputs: a startup timer, incoming awareness messages, changes in the peer list, and a manual button.

4.1 State and refs

Name	Kind	Purpose
`coordinatorId`	useState	Rendered by the badge; null while there is no coordinator
`electionInProgress`	useState	True while a timeout is armed; shows "Electing coordinator..."
`electionLog`	useState	Up to 60 entries for the Election Log tab
`termRef`	useRef	Election round counter, incremented in `startElection`
`seqRef`	useRef	Per-sender message counter used for de-duplication
`electionTimeoutRef`	useRef	The armed setTimeout handle (null when no election is pending)
`processedRef`	useRef	Map<peerId, last seq handled>
`coordinatorIdRef`	useRef	Mirror of coordinatorId readable inside long-lived callbacks
`startElectionRef`	useRef	Lets the peer-list effect and the button call the current `startElection`

Refs mirror state because the awareness listener is created once inside an effect. A closure over `coordinatorId` from that render would be frozen at its initial value; reading `coordinatorIdRef.current` always sees the latest.

4.2 Priority

Each peer publishes `priority = doc.clientID`. Higher number wins. It is a random uint32 assigned per tab by Yjs, so no clock is involved. Two tabs drawing the same value is possible in principle but extremely unlikely, and the code does not break such a tie explicitly.

4.3 The three message types

Message	Sent by	Receiver rule (from handleMessage)
ELECTION	A peer starting an election	If my clientID is higher than the sender's: broadcast ALIVE addressed to the sender, then start my own election unless one is already armed or I am already the coordinator.
ALIVE (to X)	A higher peer answering	Only the addressee (`msg.to === localPeerId`) reacts: clear the timeout, because a higher peer is alive and will announce.
COORDINATOR	The election winner	Clear any timeout and record the sender as coordinator (`announceCoordinator`).

Every message carries `{type, from, term, seq}`. Messages from self and messages with `seq <= lastSeq` are dropped before any logic runs.

4.4 startElection and declareVictory

```
startElection():
  clearElectionTimeout(); term += 1
  higher = known peers with priority > my clientID
  if higher is empty:  declareVictory()          // nobody to ask
  else: setElectionInProgress(true)
        broadcast('ELECTION')
        timeout = setTimeout(declareVictory, 2500)

declareVictory():
  clearElectionTimeout()
  broadcast('COORDINATOR'); announceCoordinator(me)
```

The known-peers list comes from `awareness.getStates()` filtered to entries whose `priority` is a number, so a peer that has not yet published its priority is not counted.

4.5 State machine

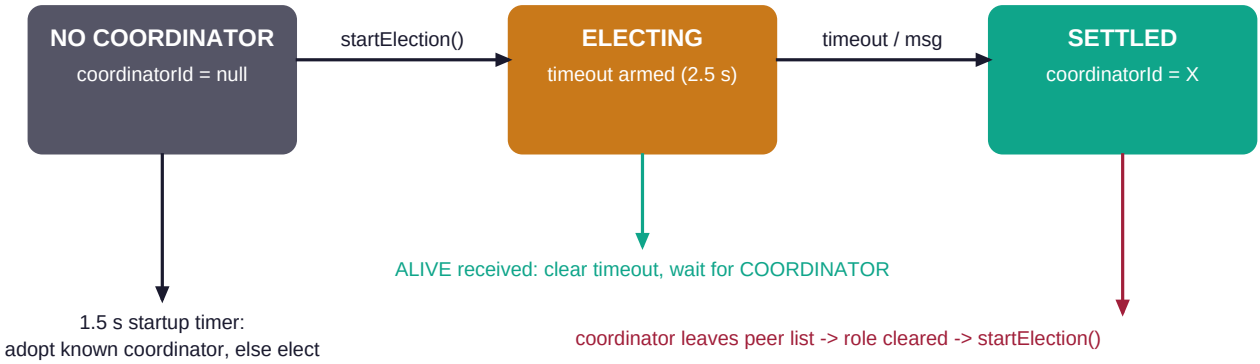

Figure 1 - Election states and the events that move between them.

4.6 The three triggers

- **Startup (1.5 s debounce).** After mount, wait `STARTUP_DEBOUNCE_MS` so the mesh can exchange awareness state. Then scan peers for a `knownCoordinator`: if found, adopt it (log: "adopted known coordinator from peer state"); otherwise call `startElection`. This is how a late joiner accepts the existing coordinator without disturbing it.
- **Coordinator loss.** A second effect diffs the `peers` array against the previous render using a Set (the same technique as `useToasts`). If the vanished ID equals `coordinatorIdRef.current`, it logs, nulls the coordinator and calls `startElectionRef.current()`. Detection speed depends on how the peer left: a closed tab removes its awareness entry on window unload, so it is near-immediate; a crash or network cut falls back to the 30 s awareness timeout.
- **Manual.** `forceReElection` logs and calls `startElection`. If the real coordinator is still the highest peer, the exchange ends with it staying coordinator.

5. Hash-Chain Audit Log: hashChain.js and useAuditLog.js

5.1 The entry format and hash

```
entry = { index, timestamp, eventType, data, prevHash, hash }

payload = `${index}|${timestamp}|${eventType}|${JSON.stringify(data)}|${prevHash}`
hash    = hex( SHA-256( UTF-8(payload) ) )     // crypto.subtle.digest
```

The hash covers every field except itself. Because `prevHash` is part of the payload, each entry commits to its predecessor's hash. Hashing uses the browser's native Web Crypto API (asynchronous, so `createEntry` and `verifyChain` are async); no library is added.

5.2 Genesis and verification

- **Genesis** (index 0) is a hard-coded constant: timestamp 0, `prevHash` of 64 zeros, and a sentinel hash of 63 zeros plus a 1. It is identical on every peer, so all chains share the same root without a network round trip. It is not a real SHA-256 output, so `verifyChain` skips it and treats it as valid.
- **verifyChain(chain)** loops from index 1: recompute the hash from the entry's own fields (`hashValid`) and check `entry.prevHash === chain[i-1].hash` (`linkValid`). It returns `{valid, results[]}`; the UI colours each row by its result.
- **What each check detects.** Editing an entry's fields breaks that entry's own `hashValid`. Replacing or reordering entries breaks `linkValid`. See Section 8.4 for an important detail about what the tamper button does and does not make fail.

5.3 Who appends, and when

Event	Trigger inside useAuditLog	Data stored
`coordinator_elected`	Effect on `isCoordinator` going false to true	coordinatorId, name
`peer_joined`	Peer-list diff finds a non-self ID not in the previous set (coordinator only)	peerId, name
`peer_left`	Peer-list diff finds a previous ID no longer present (coordinator only)	peerId
`checkpoint`	Every 5 `doc.on('update')` events while coordinator, or a 45 s interval if edits are pending	textLength, reason

The 5-update counter increments on any Y.Doc update, local or remote, while this peer is coordinator. `isCoordinatorRef` mirrors the prop for the same stale-closure reason as in Section 4.1.

5.4 appendLocalEntry

```
tail  = chainRef.current[last]
entry = await createEntry({ index: chain.length, eventType, data, prevHash: tail.hash })
chainRef.current = [...chainRef.current, entry]; setChain(chainRef.current)
seq += 1
awareness.setLocalStateField('auditEntry', { entry, seq })
```

The chain is held in a ref (so async appends always see the true tail) and mirrored into state (so React re-renders and re-verifies).

5.5 Receiving an entry

A non-coordinator accepts an `auditEntry` only when all of these hold:

- the sender's ID equals the receiver's current `coordinatorId` (entries from any other peer are ignored);
- `payload.seq` is greater than the last sequence processed for that sender;
- `entry.index === chain.length` and `entry.prevHash === tail.hash`.

If the last condition fails, the entry is dropped with a console warning and no resync is attempted (see Sections 8.2 and 8.3). The receiver does *not* recompute the new entry's SHA-256 on arrival; the verification effect checks the full chain whenever it changes.

5.6 Tamper button

`tamperEntry(index)` rewrites that entry's `data` in this peer's local chain only (adds `tampered: true`) and re-renders. Nothing is broadcast, so other peers' chains stay intact. It exists to demonstrate detection, and cannot model a real attack on the shared log.

6. UI Components

CoordinatorBadge	Pure view. Shows "Electing coordinator..." while `electionInProgress`, "Coordinator: name" (with a YOU tag when `isCoordinator`) once settled, and a force re-election button. The coordinator name is resolved in App.jsx by looking the ID up in the presence list, falling back to the local name or `Peer id`.
DebugPanel	A toggle button and two tabs. The Vector Clocks tab renders each entry with a short peer clock (last 4 digits of each ID), time, and a relation badge; the Election Log tab renders timestamped protocol lines. No logic: it only displays arrays handed in by the hooks.
AuditLogPanel	Builds a `Map<index, result>` from `verification.results` and styles each row valid or invalid. Shows truncated hashes via `truncateHash` (first 8, last 6 hex chars), the entry data, and a tamper button on every non-genesis row. The toggle shows a warning when the chain is invalid.

7. Data Flow and Trust Boundary Summary (Part 2)

Data	Path	Notes
Vector clock snapshots	Awareness field `vc` over RTCDataChannel / BroadcastChannel	Never through the signaling server; never inside the Y.Doc
Election messages	Awareness fields `coordMsg`, `priority`, `knownCoordinator`	Same channel; priority derived from Yjs clientID, not wall clock
Audit entries	Awareness field `auditEntry` from the coordinator	Replicated into each peer's React state; no server stores the chain
Audit chain at rest	Not persisted	Held in React state only, so a reloaded peer starts again from genesis
Document text	Y.Doc sync protocol (type 0)	Unchanged from Part 1; Part 2 only reads its length for checkpoints
SDP / ICE	Signaling server	The only traffic the server sees, as in Part 1

8. Bugs, Races and Edge Cases

Section 8.1 is a bug found by running the app and then fixed. Section 8.4 was checked by running the project's own `hashChain.js` under Node. Sections 8.2, 8.3, 8.5 and 8.6 come from **reading the code** and have **not** been reproduced in a live multi-tab test, so they describe consequences of the logic, not observed failures.

8.1 Fixed: a settled coordinator re-electing itself (tested)

An already-elected coordinator could receive a stray, late ELECTION from a lower-priority peer. The handler answered ALIVE (correct) and then unconditionally started its own election, which reset the coordinator state and bounced the session through an election again. The fix is the guard in the ELECTION branch:

```
if (doc.clientID > fromPriority) {
  broadcast('ALIVE', { to: fromPeerId })
  if (!electionTimeoutRef.current && coordinatorIdRef.current !== localPeerId) startElection()
}
```

8.2 From code reading: late joiners cannot backfill the audit chain

Awareness holds only the *latest* `auditEntry` per peer. A peer that joins after the chain has grown has a local chain of length 1 (genesis), but the coordinator's latest entry has a higher index, so the `entry.index === chain.length` check fails and every later entry is skipped with a warning. The README describes this gap as a missed broadcast; by the code it applies to any peer that joins after more than one entry exists: its chain holds only the genesis block, so it accepts the latest entry only if that entry has index 1.

8.3 From code reading: a new coordinator may fork the chain

A newly elected coordinator appends on top of *its own* local chain tail. If that chain is shorter or different from what other peers hold, they reject its entries (index or prevHash mismatch) and the chains diverge. This follows from the single-writer design having no state transfer on coordinator change.

8.4 Checked: the tamper test flags only the tampered entry

Entry `i` is judged by (a) recomputing its hash from its own fields and (b) comparing its `prevHash` with the *stored* `hash` of entry `i-1`. The tamper button changes entry 2's `data` but leaves its stored `hash` alone. So entry 2 fails check (a), while entry 3 still passes both checks: its own fields are unchanged and entry 2's stored hash still matches its `prevHash`. To confirm, a 5-entry chain was built with the project's `createEntry`, then entry 2 was modified the same way the button does:

```
before tamper: [ true, true, true, true, true ]
after tamper:  [ true, true, false, true, true ]     // only index 2 invalid
```

Result: a reader might expect every entry after the tampered one to turn red as well. As the code stands, only the tampered entry is flagged. A true cascade would need `verifyChain` to recompute entry `i-1`'s hash before comparing it to `prevHash`, or to mark every entry after the first failure as invalid. Hash chains are still tamper-*evident*: an attacker who edits an entry and also recomputes its hash would break the next link, but the demo button does not do that.

8.5 From code reading: message ordering inside one handler

On receiving an ELECTION, a higher peer broadcasts ALIVE and, if it then starts its own election, broadcasts ELECTION in the same call stack. Both go out as separate awareness messages because y-webrtc broadcasts on every local update, and each has its own `seq`. Correctness therefore relies on that per-update broadcast; if awareness updates were ever batched, the second value would overwrite the first

and the ALIVE could be lost.

8.6 From code reading: ALIVE without a following COORDINATOR

A settled coordinator that receives ELECTION replies ALIVE but, because of the guard in 8.1, does not start a new election, so no new COORDINATOR follows. A lower peer forcing a re-election while still holding that same coordinator ends correctly. A lower peer whose own `coordinatorId` is null would clear its timeout and wait; it relies on the 1.5 s startup adoption having already picked up the existing coordinator.

9. Constants Cheat Sheet

Constant	Value	File	Role
`ELECTION_TIMEOUT_MS`	2500 ms	useCoordinatorElection.js	Wait for an ALIVE before declaring victory
`STARTUP_DEBOUNCE_MS`	1500 ms	useCoordinatorElection.js	Let the mesh settle before adopting or electing
`MAX_LOG_ENTRIES`	60	useVectorClockLog.js, useCoordinatorElection.js	Cap on in-memory log rows
`CHECKPOINT_EVERY_EDITS`	5	useAuditLog.js	Doc updates per checkpoint entry
`CHECKPOINT_INTERVAL_MS`	45000 ms	useAuditLog.js	Time-based checkpoint if edits are pending
`CONNECT_TIMEOUT_MS`	8000 ms	useYjsRoom.js (Part 1)	Error-state heuristic
`outdatedTimeout`	30000 ms	y-protocols (library)	Crash detection for a silent peer

See SyncQuest_Part2_Project_Report.pdf for the problem statement, usage and testing summary; SyncQuest_Technical_Deep_Dive.pdf for the Part 1 internals (handshake, Y.Text merging, the two Part 1 race bugs); and README.md for setup.